



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

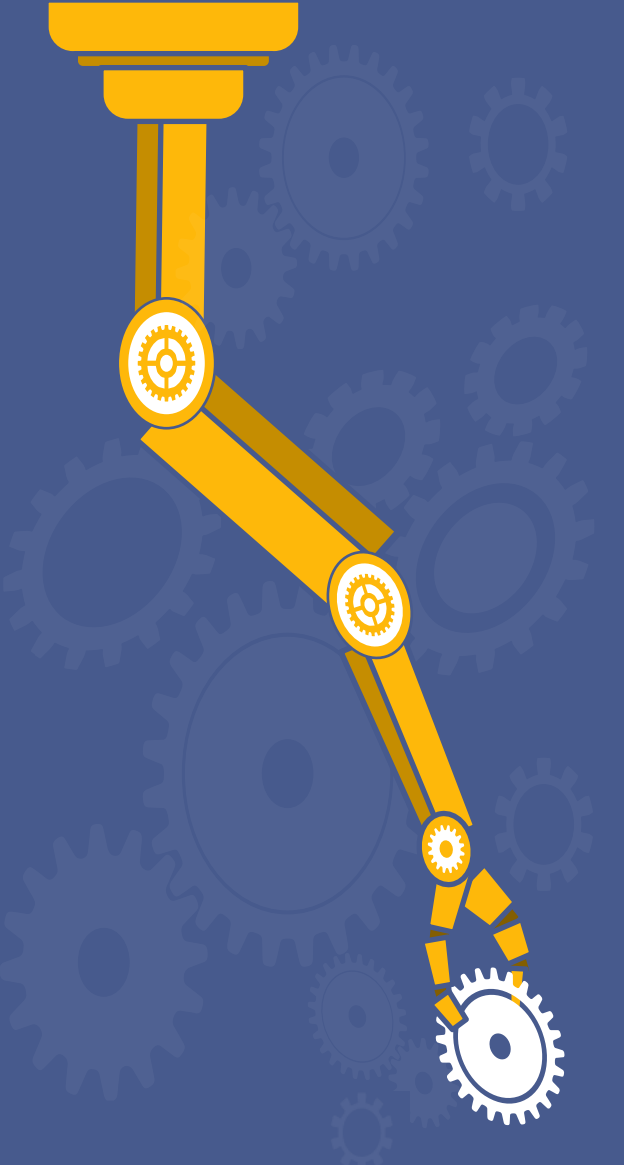
BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, <u>Floyd-Warshall Algorithm</u> - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours



BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos, Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



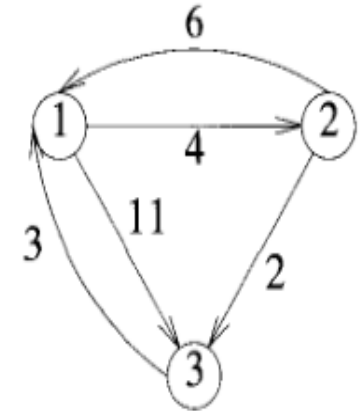
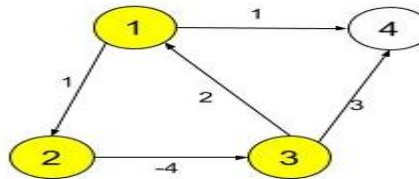
Module 4: Graph Algorithms(6 Hours)

- Single Source Shortest Algorithm
 - Bellman Ford Algorithm
- All Pair Shortest Algorithm
 - **Floyd - Warshall Algorithm**
- Network Flows – Flow Networks
- Maximum Flow
 - Ford Fulkerson
 - Edmond Karp
 - Push Re-label Algorithm



All Pair Shortest Path- Floyd Warshall

- Problem of finding shortest paths between all pairs of vertices in a directed graph $G=(V,E)$.
- **Graph:** may contain negative edges but no negative cycles.



Representation: a weight matrix where

$W(i,j)=0$ if $i=j$.

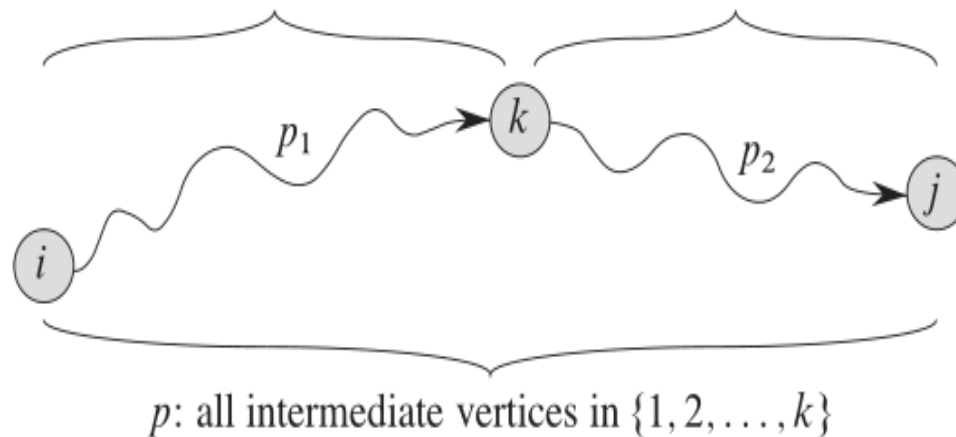
$W(i,j)=\infty$ if there is no edge between i and j .

$W(i,j)$ ="weight of edge"

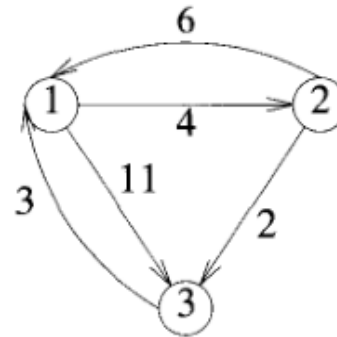


All Pair Shortest Path- Floyd Warshall

- Considers the intermediate vertices of a shortest path, where an intermediate vertex of a simple path $p=(v_1, v_2, \dots, v_n)$ is any vertex of p other than v_1 or v_n , that is, any vertex in the set $(v_2, v_3, \dots, v_{n-1})$.



All Pair Shortest Path- Floyd Warshall



A^0	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

(b) A^0

A^1	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

(c) A^1

A^2	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

(d) A^2

A^3	1	2	3
1	0	4	6
2	5	0	2
3	3	7	0

(e) A^3



All Pair Shortest Path- Floyd Warshall – Recursive solution

- Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j for which all intermediate vertices are in the set $(1, 2, \dots, k)$.
- When $k = 0$, a path from vertex i to vertex j with no intermediate vertex numbered higher than 0 has no intermediate vertices at all. Such a path has at most one edge, and hence $d_{ij}^{(0)} = w_{ij}$. **$d_{ij}^{(k)}$ is recursively defined as**

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0, \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k \geq 1. \end{cases}$$

All Pair Shortest Path- Floyd Warshall

Algorithm AllPaths(*cost*, *A*, *n*)

// *cost*[1 : *n*, 1 : *n*] is the cost adjacency matrix of a graph with
// *n* vertices; *A*[*i*, *j*] is the cost of a shortest path from vertex
// *i* to vertex *j*. *cost*[*i*, *i*] = 0.0, for $1 \leq i \leq n$.

{

 for *i* := 1 to *n* do

 for *j* := 1 to *n* do

A[*i*, *j*] := *cost*[*i*, *j*]; // Copy *cost* into *A*.

 for *k* := 1 to *n* do

 for *i* := 1 to *n* do

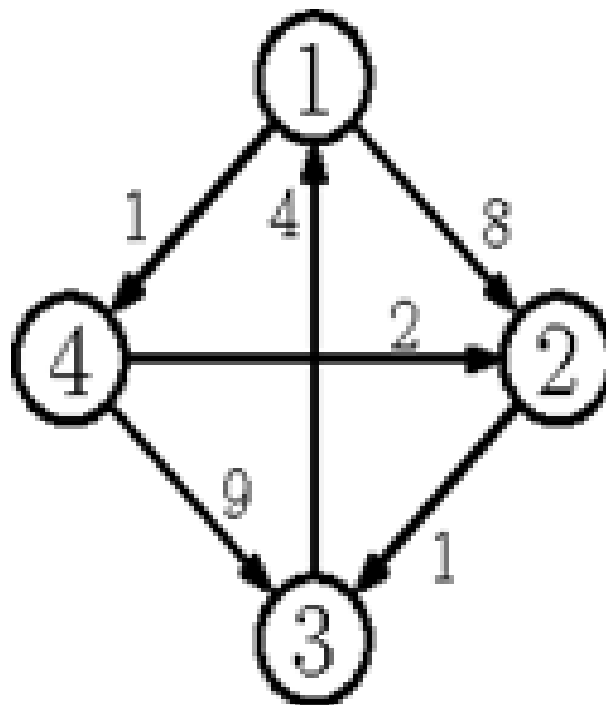
 for *j* := 1 to *n* do

A[*i*, *j*] := min(*A*[*i*, *j*], *A*[*i*, *k*] + *A*[*k*, *j*]);

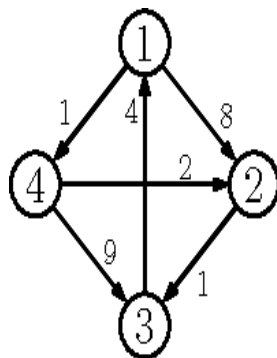
}

Time Analysis: $O(n^3)$

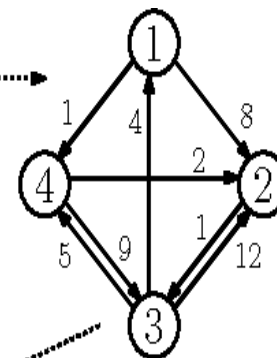
Practice



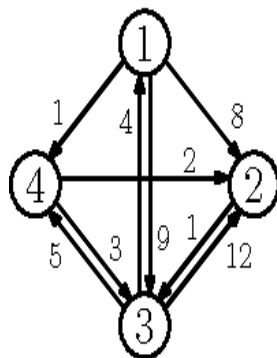
Practice



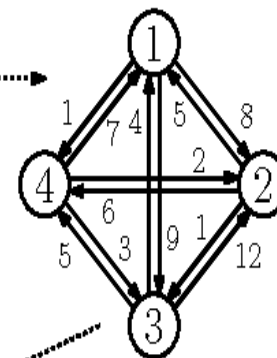
$$d^{(0)} = \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix}$$



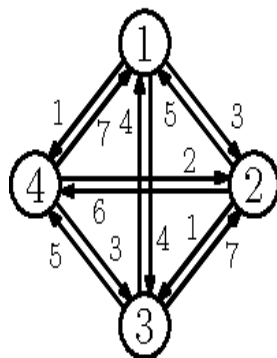
$$d^{(1)} = \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix}$$



$$d^{(2)} = \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 3 & 0 \end{bmatrix}$$



$$d^{(3)} = \begin{bmatrix} 0 & 8 & 9 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 12 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$



$$d^{(4)} = \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$

$$d^{final} = \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$